

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 7: Vision Transformer

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu nguyên lý hoạt động của Vision Transformer
- So sánh CNN và Transformer trong bài toán thị giác máy tính
- Xây dựng pipeline huấn luyện Vision Transformer
- Thực hiện classification bằng mô hình ViT

2 Giới thiệu

Vision Transformer (ViT) là một kiến trúc Deep Learning sử dụng Transformer cho bài toán thị giác máy tính.

Khác với CNN sử dụng convolution để trích xuất đặc trưng, Vision Transformer chia ảnh thành các patch và xử lý chúng như các token trong bài toán NLP.

Kiến trúc này được giới thiệu trong bài báo:

- "An Image is Worth 16x16 Words: Transformers for Image Recognition"

Vision Transformer hiện được sử dụng rộng rãi trong:

- image classification

- object detection
- semantic segmentation

3 Pipeline của Vision Transformer

Pipeline của Vision Transformer gồm các bước:

1. Chia ảnh thành các patch nhỏ
2. Chuyển mỗi patch thành vector embedding
3. Thêm positional embedding
4. Đưa chuỗi patch vào Transformer Encoder
5. Sử dụng CLS token để thực hiện classification

Pipeline tổng quát:

Image $\rightarrow$ Patch $\rightarrow$ Patch Embedding $\rightarrow$ Transformer Encoder $\rightarrow$
Classification Head

4 Dataset

Trong bài thực hành này có thể sử dụng:

- CIFAR-10
- CIFAR-100
- Flowers102

Ví dụ dataset CIFAR-10 gồm 10 lớp:

- airplane
- automobile
- bird
- cat
- deer
- dog
- frog
- horse
- ship
- truck

5 Data Augmentation

Data augmentation giúp tăng kích thước dataset và cải thiện khả năng tổng quát của mô hình.

Ví dụ:

```
transform = transforms.Compose([

    transforms.Resize((224,224)),

    transforms.RandomHorizontalFlip(),

    transforms.RandomCrop(224, padding=4),

    transforms.ToTensor()

])
```

6 Tải Dataset

```
import torchvision
import torchvision.transforms as transforms

train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)
```

7 DataLoader

```
from torch.utils.data import DataLoader
```

```

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32
)

```

8 Patch Embedding

Vision Transformer chia ảnh thành các patch nhỏ.

Ví dụ:

- ảnh 224x224
- patch size 16x16

Số patch:

$$(224/16)^2 = 196$$

Mỗi patch được biến đổi thành một vector embedding.

9 Minh họa chia ảnh thành Patch

```

import torch

def create_patches(images, patch_size=16):

    batch_size, channels, height, width = images.shape

    patches = images.unfold(2, patch_size, patch_size) \
        .unfold(3, patch_size, patch_size)

    patches = patches.contiguous().view(
        batch_size,

```

```

        channels,
        -1,
        patch_size,
        patch_size
    )

    return patches

```

10 Transformer Encoder

Vision Transformer sử dụng các Transformer Encoder giống như trong NLP.

Mỗi encoder gồm:

- Multi-head self-attention
- Feed-forward network
- Layer normalization

11 Sử dụng Vision Transformer từ thư viện

```

import torch
import torchvision.models as models

model = models.vit_b_16(pretrained=True)

```

Thay đổi lớp cuối cho CIFAR-10.

```

model.heads.head = torch.nn.Linear(
    model.heads.head.in_features,
    10
)

```

12 Training Pipeline

```

device = torch.device("cuda" if torch.cuda.is_available()
    else "cpu")

model = model.to(device)

criterion = torch.nn.CrossEntropyLoss()

```

```

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.0001
)

for epoch in range(5):

    model.train()

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

```

13 Theo dõi Loss khi Training

```

for epoch in range(5):

    running_loss = 0

    for images, labels in train_loader:

        outputs = model(images.to(device))

        loss = criterion(outputs, labels.to(device))

        optimizer.zero_grad()

        loss.backward()

```

```

optimizer.step()

running_loss += loss.item()

print("Epoch:", epoch,
      "Loss:", running_loss)

```

14 Evaluation

```

correct = 0
total = 0

model.eval()

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs,1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

print("Accuracy:", accuracy)

```

15 Inference

```

image, label = test_dataset[0]

image = image.unsqueeze(0).to(device)

```

```

model.eval()

with torch.no_grad():

    output = model(image)

pred = torch.argmax(output)

print("Predicted class:", pred)

```

16 Visualization Prediction

```

import matplotlib.pyplot as plt

image, label = test_dataset[0]

image_batch = image.unsqueeze(0).to(device)

with torch.no_grad():

    output = model(image_batch)

pred = torch.argmax(output)

plt.imshow(image.permute(1,2,0))

plt.title("Prediction: {}".format(pred))

plt.show()

```

17 So sánh CNN và Vision Transformer

Tiêu chí	CNN	Vision Transformer
Cách trích xuất đặc trưng	Convolution	Self-attention
Dữ liệu cần thiết	ít hơn	nhiều hơn
Khả năng mở rộng	tốt	rất tốt
Hiệu quả trên dataset lớn	tốt	rất tốt

18 Bài tập

1. Tải dataset CIFAR-10.

Gợi ý:

- Sử dụng thư viện `torchvision.datasets`.
- Áp dụng các phép biến đổi cơ bản bằng `torchvision.transforms`.
- Ví dụ:

```
import torchvision.datasets as datasets
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor()
])

train_dataset = datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
```

2. Hiển thị một số ảnh trong dataset bằng `matplotlib`.

Gợi ý:

- Lấy một batch từ `DataLoader`
- Sử dụng `plt.imshow()`
- Chuyển tensor sang định dạng HWC trước khi hiển thị

Ví dụ:

```
import matplotlib.pyplot as plt

img = images[0].permute(1,2,0)
plt.imshow(img)
plt.title(labels[0])
plt.show()
```

3. Huấn luyện Vision Transformer trong 5 epoch.

Gợi ý:

- Sử dụng mô hình `vit_b_16` từ `torchvision`.
- Thay đổi lớp phân loại cuối cùng để phù hợp với 10 lớp của CIFAR-10.
- Sử dụng optimizer Adam.

Ví dụ:

```
model = models.vit_b_16(pretrained=True)
model.heads.head = nn.Linear(model.heads.head.
                              in_features, 10)
```

4. Tính accuracy trên tập test.

Gợi ý:

- Chuyển mô hình sang chế độ evaluation bằng `model.eval()`
- Tắt gradient bằng `torch.no_grad()`
- So sánh nhãn dự đoán với nhãn thật

Ví dụ:

```
correct += (predicted == labels).sum().item()
accuracy = correct / total
```

5. Thay đổi learning rate và quan sát sự thay đổi accuracy.

Gợi ý:

- Thử các giá trị learning rate khác nhau, ví dụ:
 - 0.0001
 - 0.0003
 - 0.001
- So sánh accuracy sau khi huấn luyện.

6. Thử batch size khác nhau (16, 32, 64).

Gợi ý:

- Thay đổi tham số `batch_size` khi tạo `DataLoader`.
- Quan sát:
 - tốc độ huấn luyện
 - độ ổn định của loss

– accuracy cuối cùng

7. So sánh kết quả giữa Vision Transformer và một mô hình CNN đơn giản.

Gợi ý:

- Xây dựng một CNN nhỏ với vài lớp convolution.
- Ví dụ kiến trúc đơn giản:

```
class SimpleCNN(nn.Module):

    def __init__(self):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(3,32,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32,64,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Linear(64*56*56,10)

    def forward(self,x):

        x = self.conv(x)
        x = x.view(x.size(0),-1)
        x = self.fc(x)

        return x
```

8. Thử dataset CIFAR-100.

Gợi ý:

- Sử dụng `torchvision.datasets.CIFAR100`.
- Thay đổi số lớp đầu ra của mô hình thành 100.

9. Thử sử dụng mô hình ViT khác như `vit_b_32`.

Gợi ý:

- Thay đổi model:

```
model = models.vit_b_32(pretrained=True)
```

- So sánh accuracy với vit_b_16.

10. Phân tích các trường hợp mô hình dự đoán sai.

Gợi ý:

- Lưu lại các ảnh mà mô hình dự đoán sai.
- Hiển thị ảnh cùng với:
 - nhãn thật
 - nhãn dự đoán
- Thảo luận tại sao mô hình có thể nhầm lẫn.